



NPTEL Course on “Computer System Optimization for ML”

Lecture #6

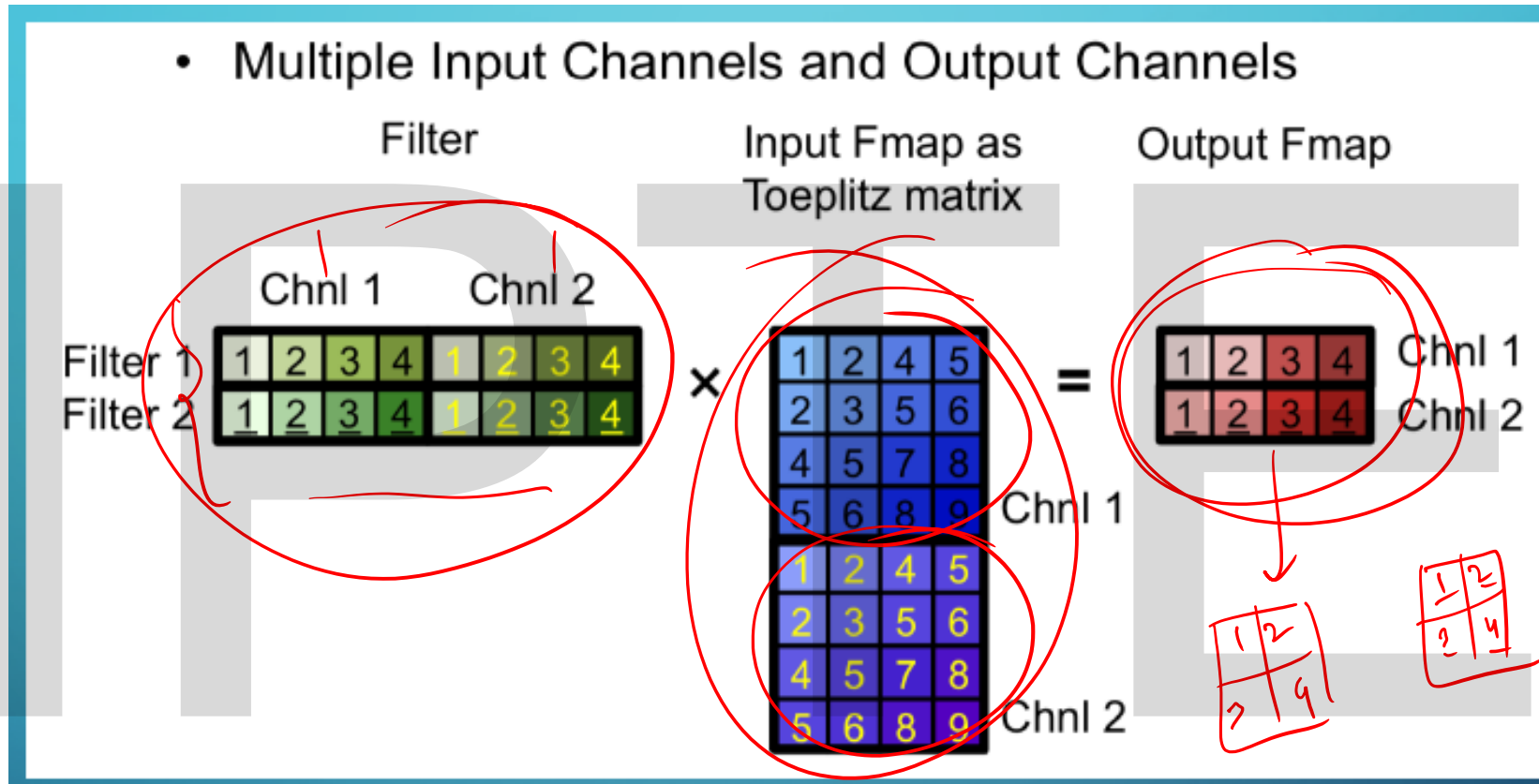


Agenda

- **Recap**
- **Quantitative Example of Flattening**
- **Fully Connected Layers**

NPTEL

Conv Layer Computation



- We can flatten both filters and input feature maps in the same way as before
- Naturally generate flattened output feature maps of two channels



NPTEL

A Quantitative Example



An Example

```
[ 1.,  2.,  3.,  4.]  
[ 5.,  6.,  7.,  8.]  
[ 9., 10., 11., 12.]  
[13., 14., 15., 16.]
```

A single channel 4×4 image

- kernel size: 2×2
- padding: 0
- stride: 1
- bias: 0
- output channels: 1
- initial weights, W : $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

Image Patch 1	Image Patch 2	Image Patch 3
[1., 2., 3., 4.] [5., 6., 7., 8.] [9., 10., 11., 12.] [13., 14., 15., 16.]	[1., 2., 3., 4.] [5., 6., 7., 8.] [9., 10., 11., 12.] [13., 14., 15., 16.]	[1., 2., 3., 4.] [5., 6., 7., 8.] [9., 10., 11., 12.] [13., 14., 15., 16.]
Image Patch 4	Image Patch 5	Image Patch 6
[1., 2., 3., 4.] [5., 6., 7., 8.] [9., 10., 11., 12.] [13., 14., 15., 16.]	[1., 2., 3., 4.] [5., 6., 7., 8.] [9., 10., 11., 12.] [13., 14., 15., 16.]	[1., 2., 3., 4.] [5., 6., 7., 8.] [9., 10., 11., 12.] [13., 14., 15., 16.]
Image Patch 7	Image Patch 8	Image Patch 9
[1., 2., 3., 4.] [5., 6., 7., 8.] [9., 10., 11., 12.] [13., 14., 15., 16.]	[1., 2., 3., 4.] [5., 6., 7., 8.] [9., 10., 11., 12.] [13., 14., 15., 16.]	[1., 2., 3., 4.] [5., 6., 7., 8.] [9., 10., 11., 12.] [13., 14., 15., 16.]

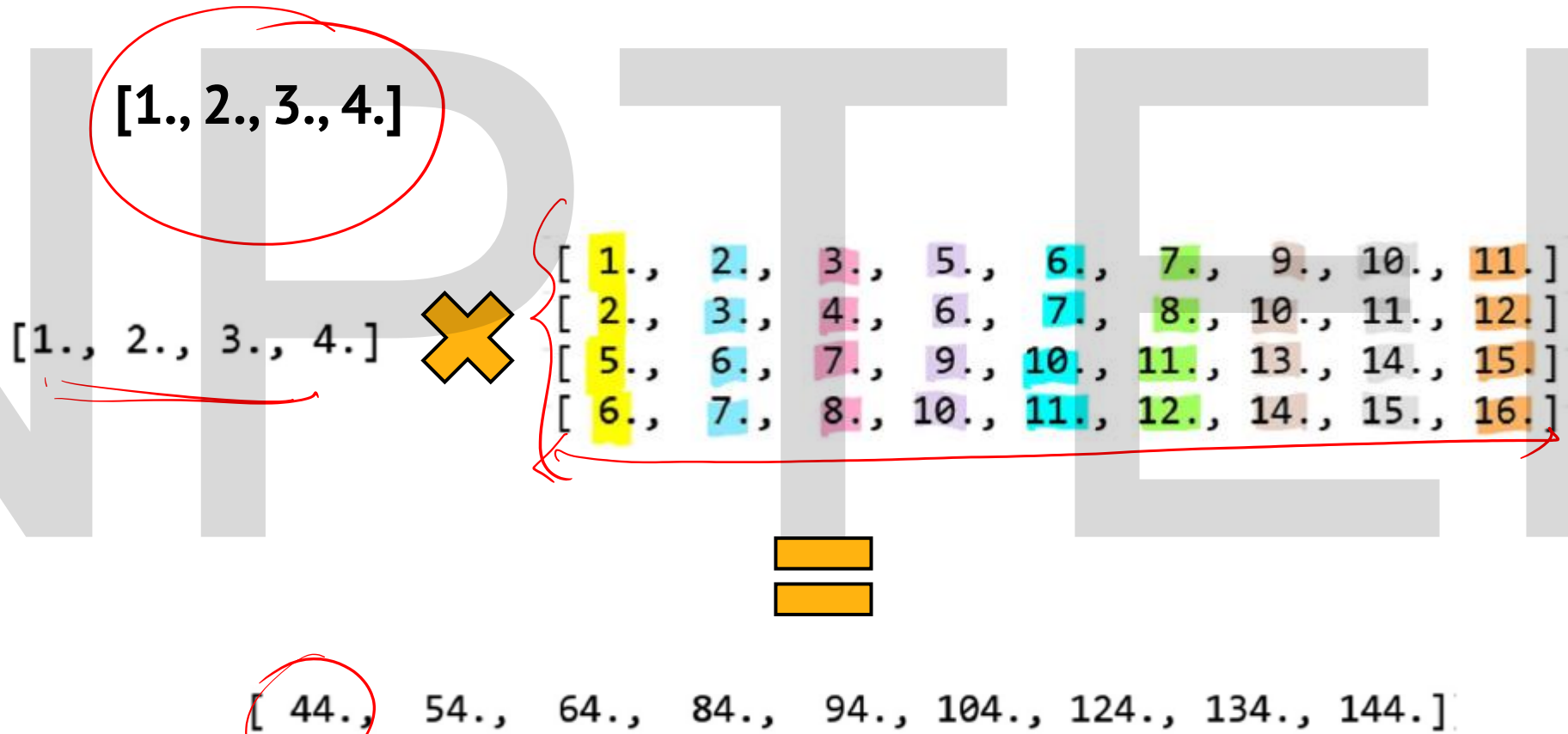
Converting Filter Traces into Matrix

Image Patch 1	Image Patch 2	Image Patch 3
[1., 2., 3., 4.]	[1., 2., 3., 4.]	[1., 2., 3., 4.]
[5., 6., 7., 8.]	[5., 6., 7., 8.]	[5., 6., 7., 8.]
[9., 10., 11., 12.]	[9., 10., 11., 12.]	[9., 10., 11., 12.]
[13., 14., 15., 16.]	[13., 14., 15., 16.]	[13., 14., 15., 16.]
Image Patch 4	Image Patch 5	Image Patch 6
[1., 2., 3., 4.]	[1., 2., 3., 4.]	[1., 2., 3., 4.]
[5., 6., 7., 8.]	[5., 6., 7., 8.]	[5., 6., 7., 8.]
[9., 10., 11., 12.]	[9., 10., 11., 12.]	[9., 10., 11., 12.]
[13., 14., 15., 16.]	[13., 14., 15., 16.]	[13., 14., 15., 16.]
Image Patch 7	Image Patch 8	Image Patch 9
[1., 2., 3., 4.]	[1., 2., 3., 4.]	[1., 2., 3., 4.]
[5., 6., 7., 8.]	[5., 6., 7., 8.]	[5., 6., 7., 8.]
[9., 10., 11., 12.]	[9., 10., 11., 12.]	[9., 10., 11., 12.]
[13., 14., 15., 16.]	[13., 14., 15., 16.]	[13., 14., 15., 16.]

[1., 2., 3., 5., 6., 7., 9., 10., 11.]
[2., 3., 4., 6., 7., 8., 10., 11., 12.]
[5., 6., 7., 9., 10., 11., 13., 14., 15.]
[6., 7., 8., 10., 11., 12., 14., 15., 16.]

Filter Flattened into a Vector

- The matrix of weights for the convolutional layer can be flattened into a vector, K



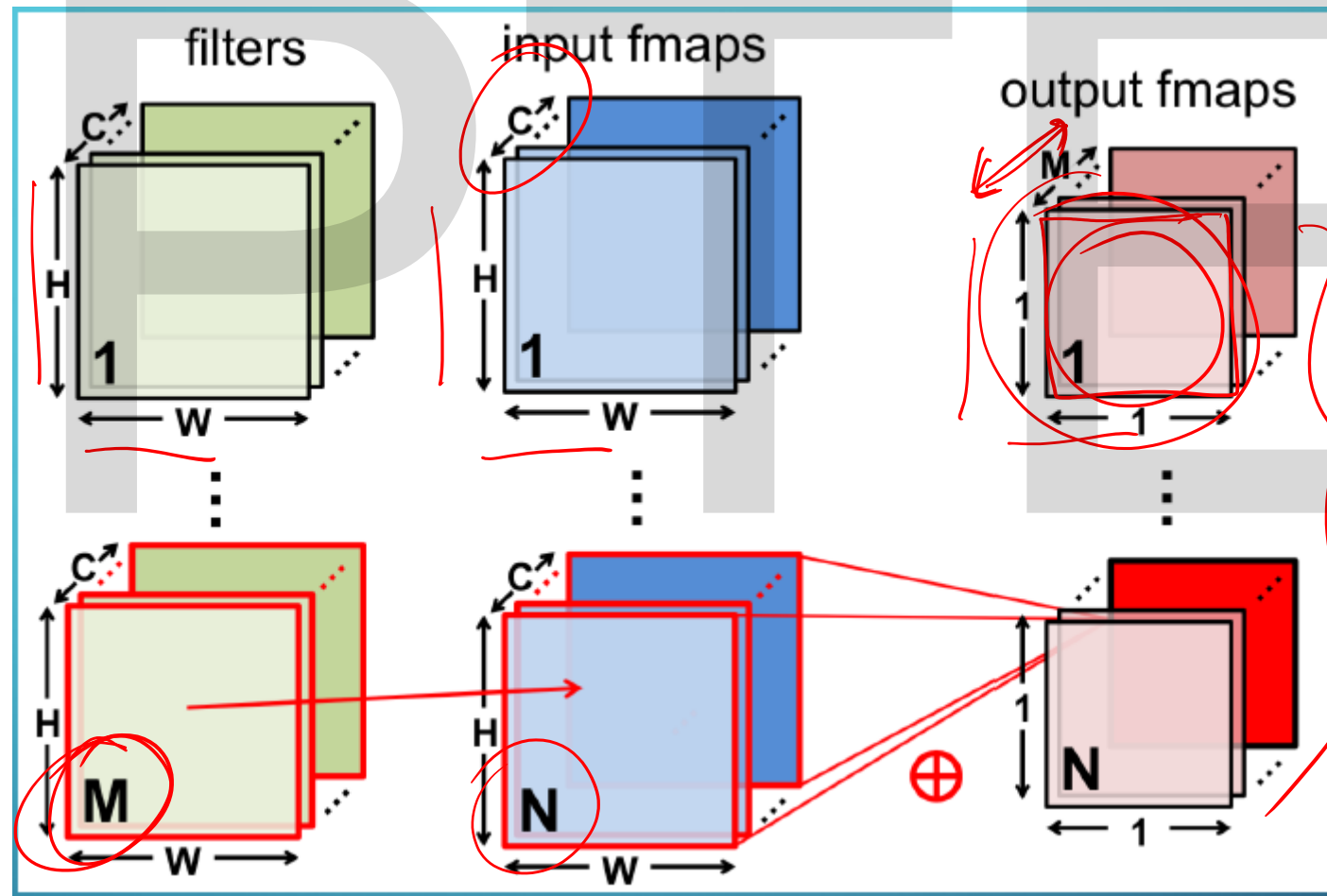


Fully Connected Layer: Extracting Parallelism

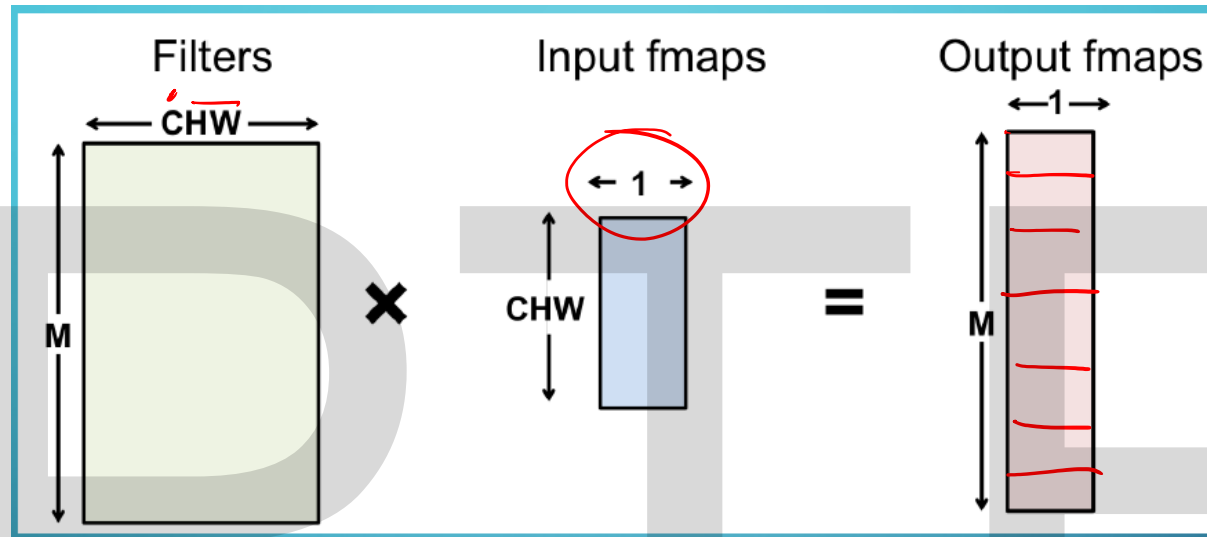


Batch of N Input fmaps

- A batch has N input fmaps, so we apply the same M filters of size CHW, and generate N output fmaps of 1x1xM



Fully Connected Layer

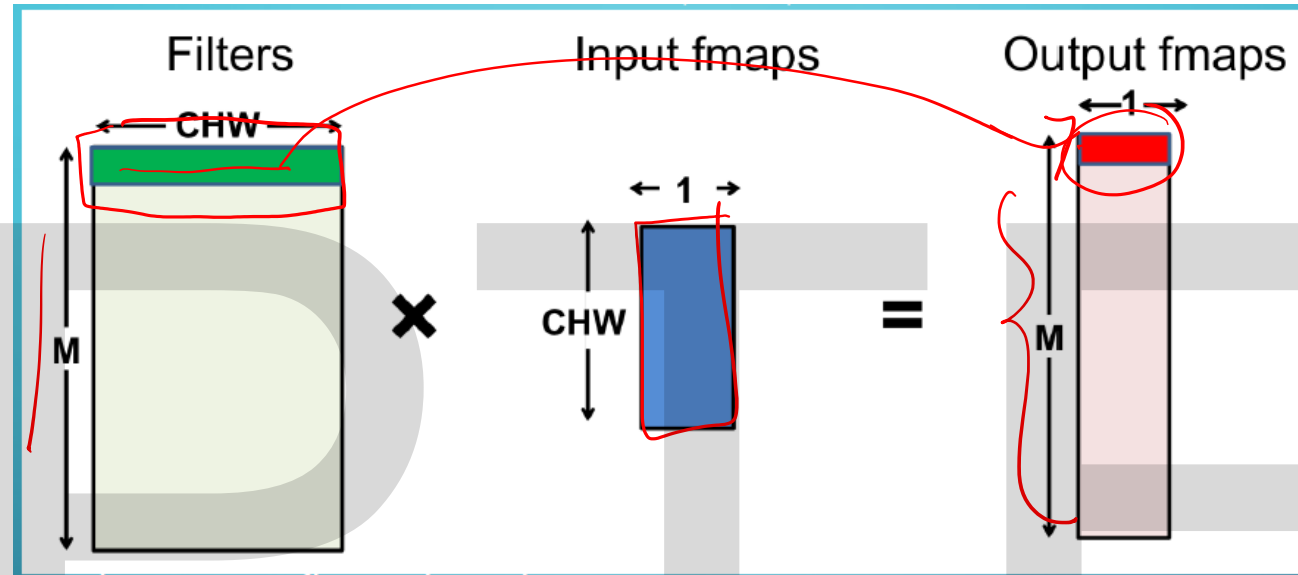


$$[M \times CHW] \times [CHW \times 1] = [M \times 1]$$

rows

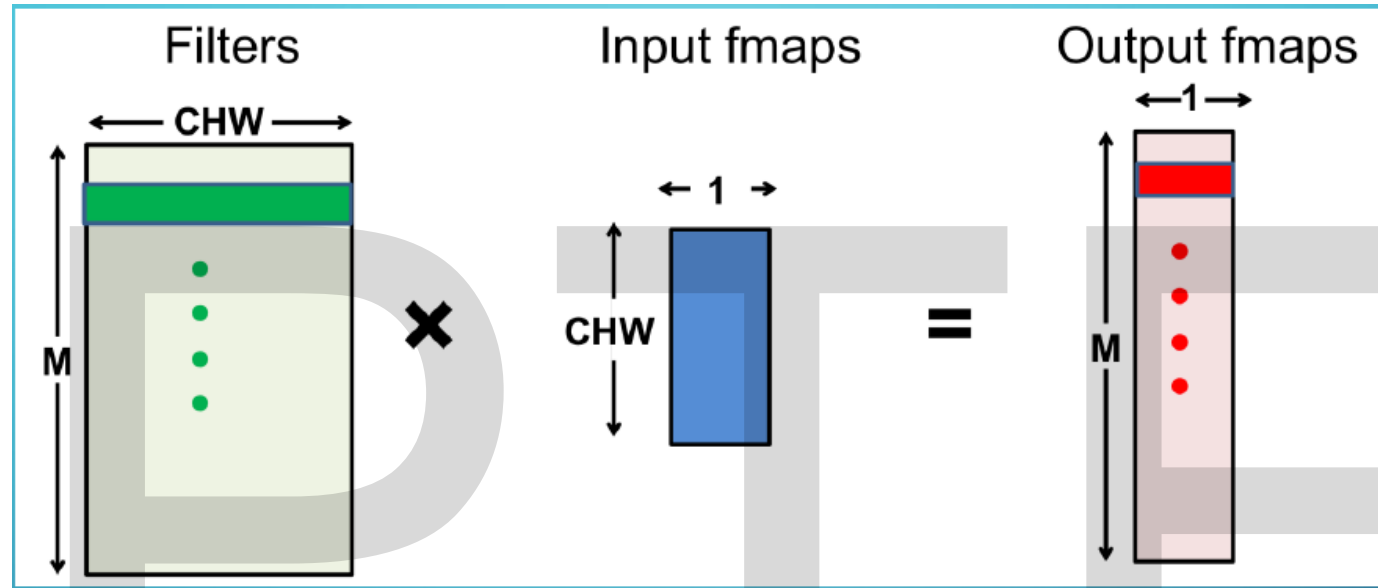
- We flatten three dimensions into one
 - C (#of channels input fmap), HxW (filter size)
 - All filters of for the input fmaps are flattened into one row of CHW. We have M such rows, each for an output channel
- To perform matrix multiplication, input fmaps also need to be flattened to have CHW rows
 - After multiplication, the output contains one point for all M output channels

Fully Connected Layer



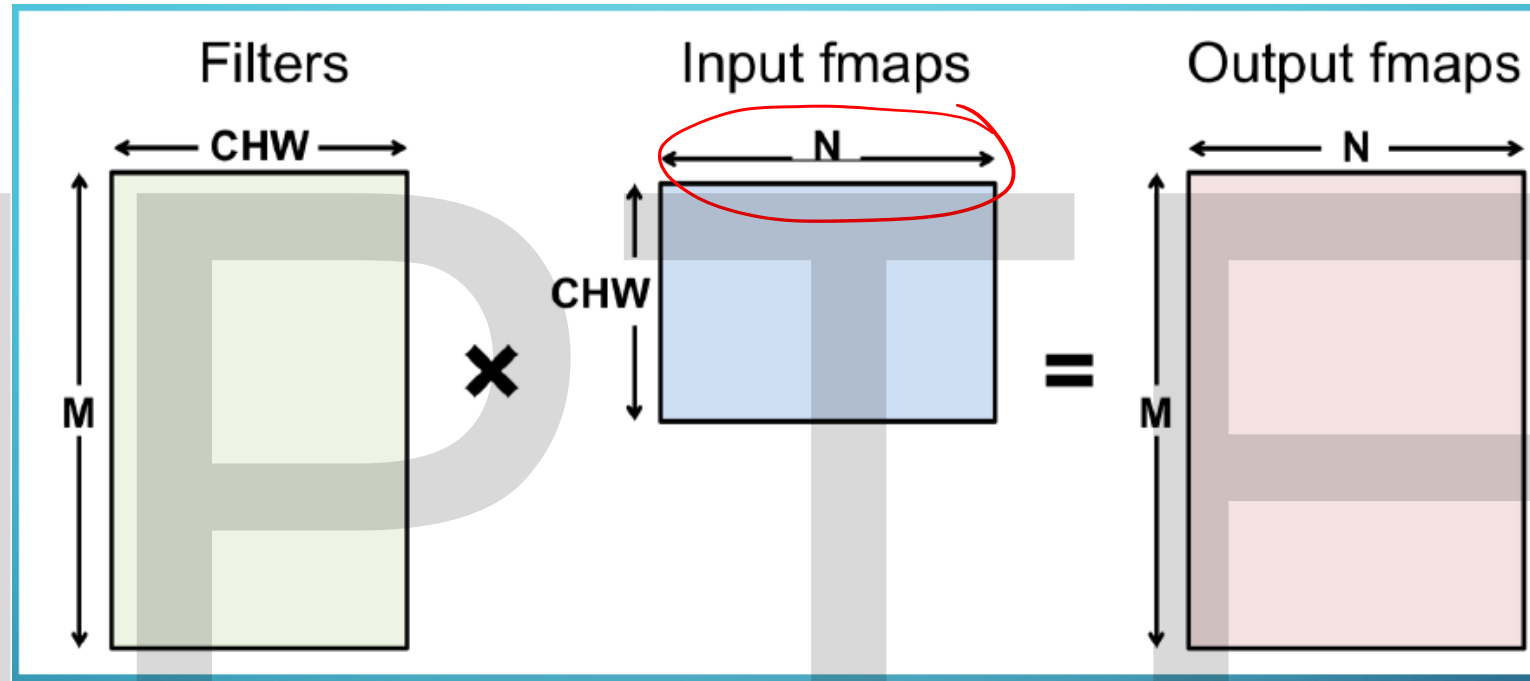
- We flatten three dimensions into one
 - C (#of channels input fmap), $H \times W$ (filter size)
 - All filters of for the input fmaps are flattened into one row of CHW . We have M such roes, each for an output channel
- To perform matrix multiplication, input fmaps also need to be flattened to have CHW rows
 - After multiplication, the output contains one point for all M output channels

Fully Connected Layer



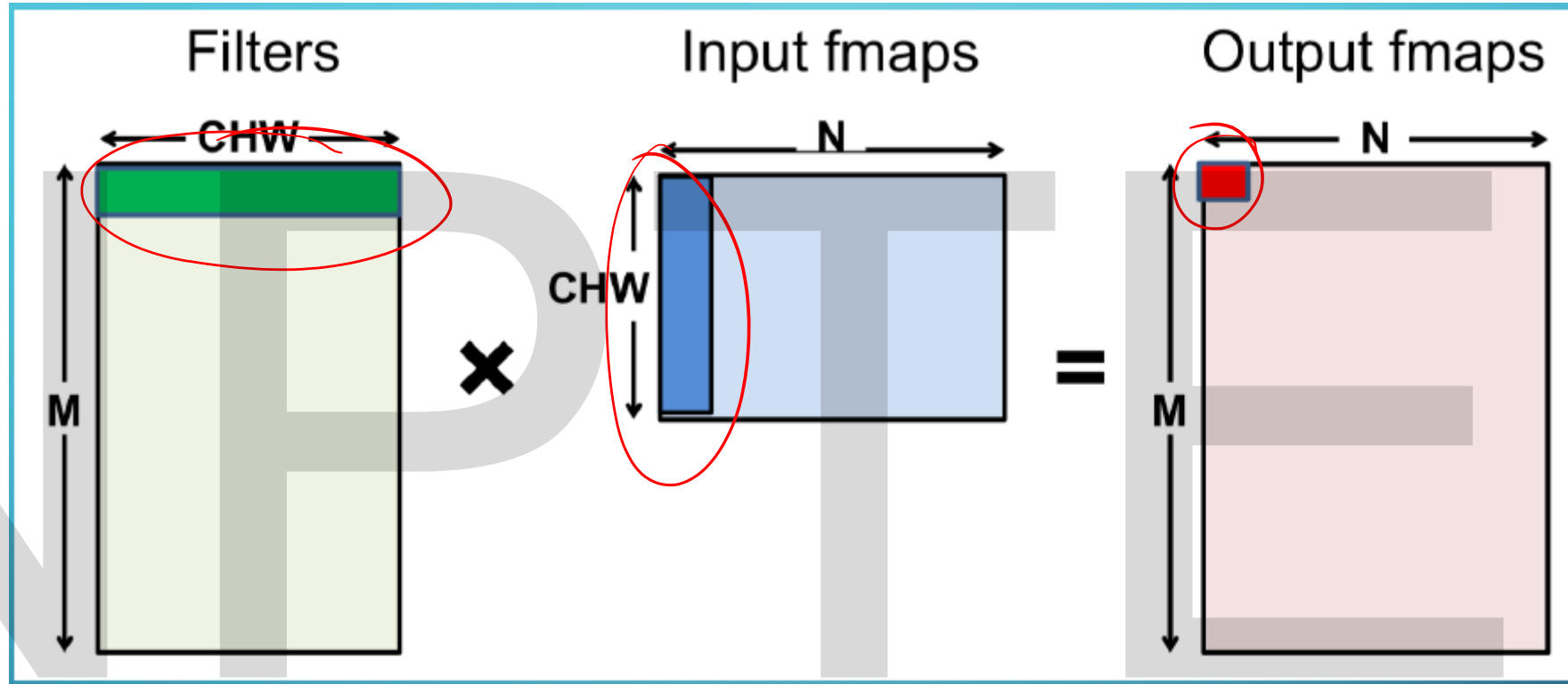
- We flatten three dimensions into one
 - C (#of channels input fmap), $H \times W$ (filter size)
 - All filters of for the input fmaps are flattened into one row of CHW . We have M such roes, each for an output channel
- To perform matrix multiplication, input fmaps also need to be flattened to have CHW rows
 - After multiplication, the output contains one point for all M output channels

Flattened Fully Connected Layer



- After flattening, having a batch size of N turns the matrix-vector operation into a matrix-matrix multiplication

Flattened Fully Connected Layer

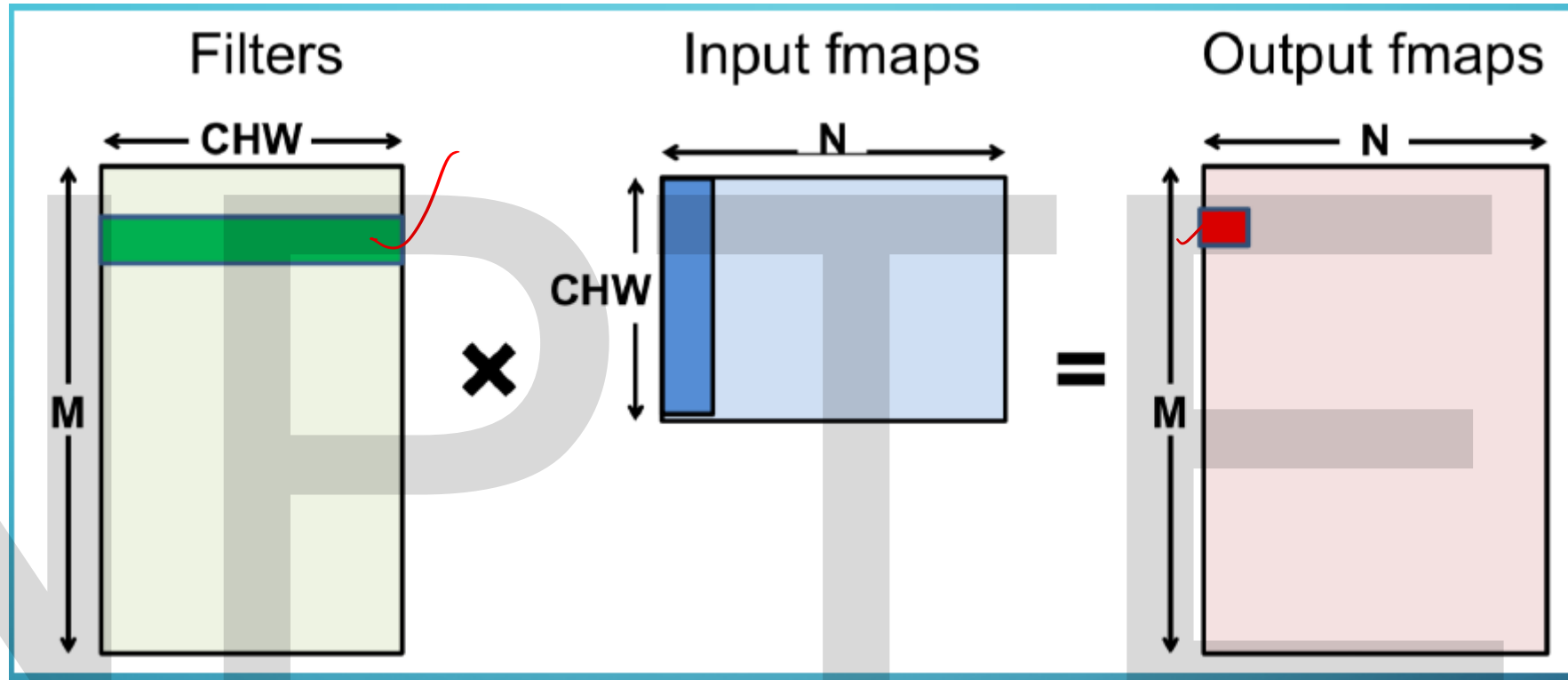


- After flattening, having a batch size of N turns the matrix-vector operation into a matrix-matrix multiplication

The diagram illustrates the block-matrix multiplication of N and P to produce L . Matrix N (green) has height M and width CHW . Matrix P (blue) has height CHW and width M . The resulting matrix L (pink) has height M and width M . A red circle highlights a specific element in L , which is the result of the dot product of a row from N and a column from P .

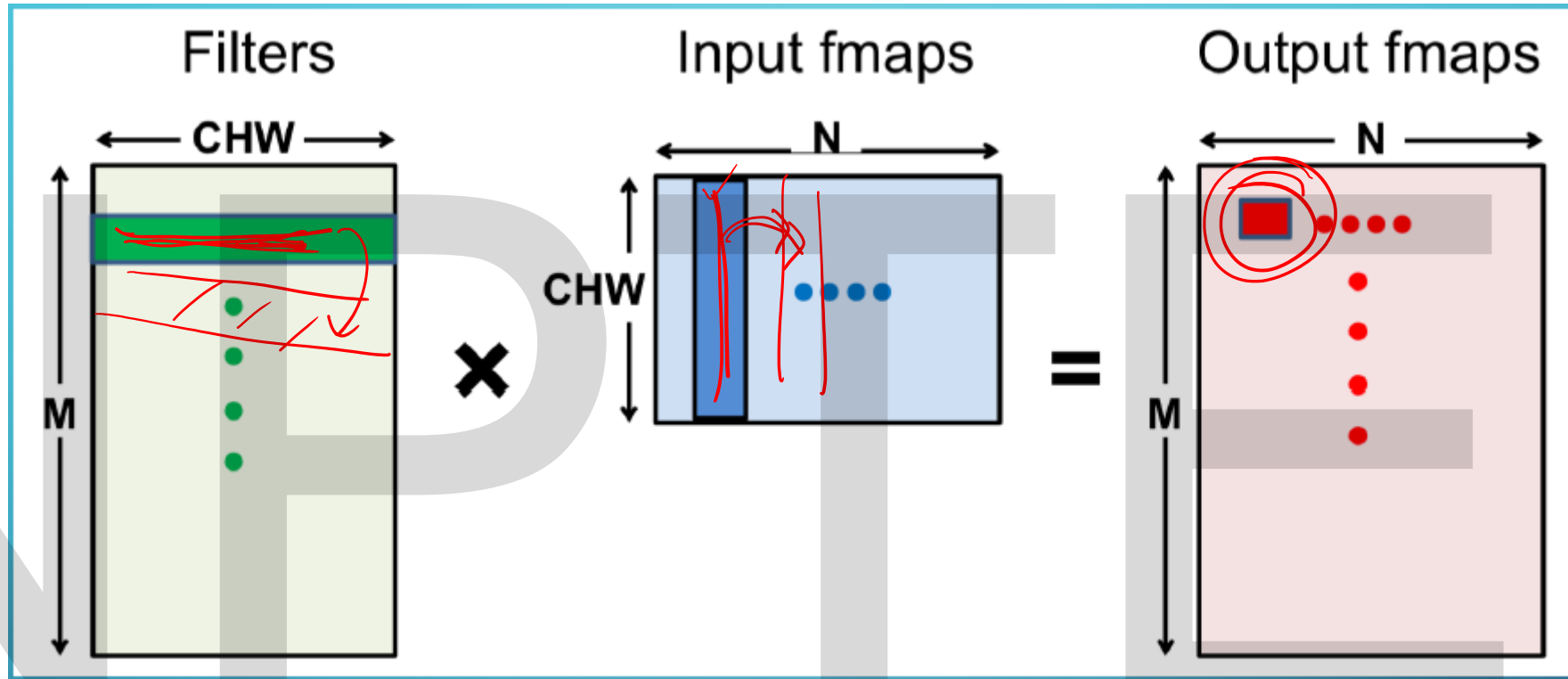
- 15

Flattened Fully Connected Layer



- After flattening, having a batch size of N turns the matrix-vector operation into a matrix-matrix multiplication

Flattened Fully Connected Layer



- After flattening, having a batch size of N turns the matrix-vector operation into a matrix-matrix multiplication
- How much temporal locality (reuse of data within a time frame) for this implementation?
 - None



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)

